Pratique TP1

2026-10-18

Exercice n°1 :

```
cat("\014")

# Question 1 :
notesVector <- c(7, 9, 13, 8, 4, 2, 16, 1, 6, 19, 15,
                 12, 19, 14, 8, 2, 19, 11, 18, 7) # c'est la fonction c()
# Question 2 :
# a :
secondElement <- notesVector[2]; secondElement

## [1] 9

# b :
fiveFirstElements <- notesVector[1:5]; fiveFirstElements

## [1]  7  9 13  8  4

# c :
great14 <- notesVector[notesVector >= 14]; great14

## [1] 16 19 15 19 14 19 18

# d :
allElements61012 <- notesVector[-c(6, 10, 12)]; allElements61012

##  [1]  7  9 13  8  4 16  1  6 15 19 14  8  2 19 11 18  7
```

Exercice n°2 :

```
cat("\014")

# Question 1 :
vecRep <- rep(c(0, 6), 3); vecRep      # répition du motif (0, 6) trois fois

## [1] 0 6 0 6 0 6

# Question 2 :
vecSeq1 <- seq(1, 10, by = 3); vecSeq1   # suite de 1 à 10 avec un pas de 3

## [1]  1  4  7 10

vecSeq1 <- seq(from = 1, to = 10, by = 3); vecSeq1
```

```
## [1]  1  4  7 10
```

```
# Question 3 :
vecRep2 <- rep(c(1, 2, 3), 4); vecRep2  # répition du motif (1, 2, 3) quatre fois
```

```
##  [1] 1 2 3 1 2 3 1 2 3 1 2 3
```

```
# Question 4 :
vecRepSeq <- rep(1:3, times = 1:3); vecRepSeq
```

```
## [1] 1 2 2 3 3 3
```

```
vecRepSeq1 <- rep(seq(1, 3), times = 1:3); vecRepSeq1
```

```
## [1] 1 2 2 3 3 3
```

```
vecRepSeq2 <- rep(c(1, 2, 3), times = c(1, 2, 3)); vecRepSeq2
```

```
## [1] 1 2 2 3 3 3
```

```
# Question 5 :
vecRepSeq3 <- rep(1:3, times = 3:1); vecRepSeq3
```

```
## [1] 1 1 1 2 2 3
```

```
# Question 6 :
vecSeq2 <- seq(from = 1, to = 10, by = 4.5); vecSeq2
```

```
## [1]  1.0  5.5 10.0
```

```
# Question 7 :
vecRep4 <- rep(1:3, each = 4); vecRep4
```

```
##  [1] 1 1 1 1 2 2 2 2 3 3 3 3
```

```
vecRep5 <- rep(1:3, times = c(4, 4, 4)); vecRep5
```

```
##  [1] 1 1 1 1 2 2 2 2 3 3 3 3
```

```
vecRep6 <- rep(1:3, times = rep(4, 3)); print(vecRep6); vecRep6
```

```
##  [1] 1 1 1 1 2 2 2 2 3 3 3 3
```

```
##  [1] 1 1 1 1 2 2 2 2 3 3 3 3
```

Exercice 3 :

```
cat("\014")
```

```
# Question 1 :
aVector <- c(2.5, 3, 1, 0, 4, -1); aVector
```

```
## [1]  2.5  3.0  1.0  0.0  4.0 -1.0
```

```
# Question 2 :
aSortedVector <- sort(aVector); aSortedVector   # sort ordonne les éléments d
ans l'ordre croissant
```

```
## [1] -1.0  0.0  1.0  2.5  3.0  4.0
```

```
aSortedVector <- sort(aVector, decreasing = TRUE); aSortedVector  # sort ordo
nne les éléments dans l'ordre décroissant
```

```
## [1]  4.0  3.0  2.5  1.0  0.0 -1.0
```

```
# La fonction order renvoie les indices des élements ordonnés
# La fonction order() renvoie les indices des éléments qui permettraient
# de trier le vecteur, et non directement les valeurs triées.
index <- order(aVector); print(index)
```

```
## [1] 6 4 3 1 2 5
```

```
aOrdredVector <- aVector[index]; aOrdredVector
```

```
## [1] -1.0  0.0  1.0  2.5  3.0  4.0
```

Execice 4 :

```
cat("\014")
```

```
# Question 1 :

observationVector <- c(7, 13, 3, 8, 12, 12, 20, 11)
sumTermes <- sum(1 / observationVector)
```

```
# Question 2 :

harmonicMean <- length(observationVector) / sumTermes

print(harmonicMean)
```

```
## [1] 8.116148
```

```
sprintf("The harmonic mean is equal to : %.2f", harmonicMean)
```

```
## [1] "The harmonic mean is equal to : 8.12"
```

```
# observationVector <- c(7, 13, 3, 8, 12, 12, 20, 11)
# harmonicMean <- length(observationVector)/sum(1 / observationVector); print
(harmonicMean)

# observationVector <- c(7, 13, 3, 8, 12, 12, 20, 11)
```

```r
#
# N <- length(observationNector)
#
# H <- N / sum(1 / observationVector)
#
# print(H)
```

Exercice 5 :

```r
# Question 1 :
cat("\014")

set.seed(123)     # pour fixer le résultat du vecteur aléatoire entre deux ex
écutions
                  # 123 Il n'a aucune signification particulière : c'est simp
lement une
                  # valeur choisie par convention ou par habitude. Vous pouve
z utiliser
                  # par exemple : 1, 42, 2026, 1000
xVector <- runif(10, min = 0, max = 1); print(xVector)
```

```
##  [1] 0.2875775 0.7883051 0.4089769 0.8830174 0.9404673 0.0455565 0.5281055
##  [8] 0.8924190 0.5514350 0.4566147
```

```r
# Question 2 :

# yVector <- c(
#   rep(0, sum(xVector < 0.5)),
#   rep(1, sum(xVector >= 0.5))
# ); print(yVector)    # l'ordre n'est pas conservé

# yVector <- numeric(length(xVector))
# yVector <- c(
#   ifelse(xVector < 0.5, 0, NA),
#   ifelse(xVector >= 0.5, 1, NA)
# ); yVector <- yVector[!is.na(yVector)] ; print(yVector) # l'ordre n'est pas
 conservé aussi

yVector <- as.numeric(xVector >= 0.5); print(yVector)
```

```
##  [1] 0 1 0 1 1 0 1 1 1 0
```

```r
# Question 3 :
# Nombre des éléments de xVector > 0.5
countGreat05 <- length(xVector[xVector > 0.5]); print(countGreat05)
```

```
## [1] 6
```

```r
# countGreat05 <- sum(xVector > 0.5)
proportionGreat05 <- countGreat05 / length(xVector) * 100
cat("Proportion > 0.5 : ", proportionGreat05, "%")
```

```
## Proportion > 0.5 :  60 %
```

```r
# Répétition de l'expérience :

# ============================================================
# Question 4 : Répétition de l'expérience
# pour différentes tailles N
# ============================================================

#set.seed(123)

tailles <- c(10, 50, 100, 500, 1000, 10000)

for (N in tailles) {

  xVector <- runif(N, min = 0, max = 1)

  countGreat05 <- sum(xVector > 0.5)

  proportionGreat05 <- countGreat05 / N

  cat("N =", N,
      "| Nombre =", countGreat05,
      "| Proportion =", proportionGreat05,
      "| Pourcentage =", proportionGreat05 * 100, "%\n")
}
```

```
## N = 10 | Nombre = 5 | Proportion = 0.5 | Pourcentage = 50 %
## N = 50 | Nombre = 23 | Proportion = 0.46 | Pourcentage = 46 %
## N = 100 | Nombre = 44 | Proportion = 0.44 | Pourcentage = 44 %
## N = 500 | Nombre = 250 | Proportion = 0.5 | Pourcentage = 50 %
## N = 1000 | Nombre = 490 | Proportion = 0.49 | Pourcentage = 49 %
## N = 10000 | Nombre = 4966 | Proportion = 0.4966 | Pourcentage = 49.66 %
```

```r
# ============================================================
# Question 5 : Répéter l'expérience 1000 fois
# pour chaque taille N
# ============================================================

#set.seed(123)

tailles <- c(10, 50, 100, 500, 1000, 10000)

nbSimulations <- 1000
```

```r
for (N in tailles) {

  proportions <- replicate(nbSimulations, {

    xVector <- runif(N, min = 0, max = 1)

    countGreat05 <- sum(xVector > 0.5)

    countGreat05 / N
  })

  cat("N =", N,
      "| Proportion moyenne =", mean(proportions),
      "| Écart-type =", sd(proportions), "\n")
}
```

```
## N = 10 | Proportion moyenne = 0.4889 | Écart-type = 0.1615949
## N = 50 | Proportion moyenne = 0.4965 | Écart-type = 0.0706905
## N = 100 | Proportion moyenne = 0.50137 | Écart-type = 0.04986291
## N = 500 | Proportion moyenne = 0.499474 | Écart-type = 0.02198059
## N = 1000 | Proportion moyenne = 0.49973 | Écart-type = 0.01606353
## N = 10000 | Proportion moyenne = 0.4999205 | Écart-type = 0.004976235
```

```r
# ============================================================
# Conclusion
# ============================================================

cat("\nLa proportion théorique est 0.5, soit 50 %.\n")
```

```
##
## La proportion théorique est 0.5, soit 50 %.
```

```r
cat("Lorsque N augmente, la proportion observée tend à se rapprocher de 0.5.\
n")
```

```
## Lorsque N augmente, la proportion observée tend à se rapprocher de 0.5.
```

```r
# Exercice 5 : Une autre version :

# ============================================================
# Simulation d'une expérience aléatoire
# Utilisation de runif(), sum(), mean() et replicate()
# ============================================================



# ------------------------------------------------------------
# 1. Créer un vecteur x_vector de taille N = 50
# ------------------------------------------------------------
cat("\014")
```

```r
set.seed(123)

N <- 50

xVector <- runif(N)

cat("1. Vecteur x_vector :\n")
```

```
## 1. Vecteur x_vector :
```

```r
print(xVector)
```

```
##  [1] 0.28757752 0.78830514 0.40897692 0.88301740 0.94046728 0.04555650
##  [7] 0.52810549 0.89241904 0.55143501 0.45661474 0.95683335 0.45333416
## [13] 0.67757064 0.57263340 0.10292468 0.89982497 0.24608773 0.04205953
## [19] 0.32792072 0.95450365 0.88953932 0.69280341 0.64050681 0.99426978
## [25] 0.65570580 0.70853047 0.54406602 0.59414202 0.28915974 0.14711365
## [31] 0.96302423 0.90229905 0.69070528 0.79546742 0.02461368 0.47779597
## [37] 0.75845954 0.21640794 0.31818101 0.23162579 0.14280002 0.41454634
## [43] 0.41372433 0.36884545 0.15244475 0.13880606 0.23303410 0.46596245
## [49] 0.26597264 0.85782772
```

```r
# ------------------------------------------------------------
# 2. Créer le vecteur yVector
#    y_i = 0 si x_i < 0.5
#    y_i = 1 si x_i >= 0.5
# ------------------------------------------------------------

yVector <- as.numeric(xVector >= 0.5)

cat("\n2. Vecteur yVector :\n")
```

```
##
## 2. Vecteur yVector :
```

```r
print(yVector)
```

```
##  [1] 0 1 0 1 1 0 1 1 1 0 1 0 1 1 0 1 0 0 0 1 1 1 1 1 1 1 1 1 0 0 1 1 1 1 0
 0 1 0
## [39] 0 0 0 0 0 0 0 0 0 0 0 1
```

```r
# Vérifier que les deux vecteurs ont la même taille

cat("\nTaille de xVector :", length(xVector), "\n")
```

```
##
## Taille de xVector : 50
```

```r
cat("Taille de yVector :", length(yVector), "\n")
```

```
## Taille de yVector : 50
```

```r
# ------------------------------------------------------------
# 3. Calculer le nombre et la proportion des valeurs
#    de xVector strictement supérieures à 0.5
# ------------------------------------------------------------

nombre <- sum(xVector > 0.5)

proportion <- nombre / N

cat("\n3. Nombre de valeurs > 0.5 :", nombre, "\n")
```

```
##
## 3. Nombre de valeurs > 0.5 : 25
```

```r
cat("Proportion :", proportion, "\n")
```

```
## Proportion : 0.5
```

```r
cat("Pourcentage :", proportion * 100, "%\n")
```

```
## Pourcentage : 50 %
```

```r
# Vérification avec mean()

proportion2 <- mean(xVector > 0.5)

cat("Proportion calculée avec mean() :", proportion2, "\n")
```

```
## Proportion calculée avec mean() : 0.5
```

```r
# ------------------------------------------------------------
# 4. Recommencer l'expérience pour différentes tailles N
# ------------------------------------------------------------

set.seed(123)

tailles <- c(10, 50, 100, 1000, 10000)

cat("\n4. Comparaison selon la taille de l'échantillon\n\n")
```

```
##
## 4. Comparaison selon la taille de l'échantillon
```

```r
for (N in tailles) {

  xVector <- runif(N)

  nombre <- sum(xVector > 0.5)

  proportion <- nombre / N
```

```r
  cat("N =", N,
      "| Nombre =", nombre,
      "| Proportion =", proportion,
      "| Pourcentage =", proportion * 100, "%\n")
}
```

```
## N = 10 | Nombre = 6 | Proportion = 0.6 | Pourcentage = 60 %
## N = 50 | Nombre = 23 | Proportion = 0.46 | Pourcentage = 46 %
## N = 100 | Nombre = 45 | Proportion = 0.45 | Pourcentage = 45 %
## N = 1000 | Nombre = 500 | Proportion = 0.5 | Pourcentage = 50 %
## N = 10000 | Nombre = 4949 | Proportion = 0.4949 | Pourcentage = 49.49 %
```

```r
# ------------------------------------------------------------
# 5. Répéter 1000 fois l'expérience pour chaque taille N
# ------------------------------------------------------------

set.seed(123)

tailles <- c(10, 50, 100, 1000, 10000)

nombre_simulations <- 1000

cat("\n5. Résultats de 1000 simulations\n\n")
```

```
##
## 5. Résultats de 1000 simulations
```

```r
for (N in tailles) {

  proportions <- replicate(nombre_simulations, {
    xVector <- runif(N)
    mean(xVector > 0.5)
  })

  cat("N =", N,
      "| Proportion moyenne =", mean(proportions),
      "| Écart-type =", sd(proportions), "\n")
}
```

```
## N = 10 | Proportion moyenne = 0.4943 | Écart-type = 0.1556333
## N = 50 | Proportion moyenne = 0.49536 | Écart-type = 0.07152892
## N = 100 | Proportion moyenne = 0.50102 | Écart-type = 0.0496349
## N = 1000 | Proportion moyenne = 0.499563 | Écart-type = 0.01567762
## N = 10000 | Proportion moyenne = 0.4999251 | Écart-type = 0.004945535
```

```r
# ------------------------------------------------------------
# 6. Conclusion
# ------------------------------------------------------------

cat("\nConclusion : la proportion observée tend vers 0.5",
```

```
    "lorsque N augmente.\n",
    "C'est une illustration de la loi des grands nombres. \n")

##
## Conclusion : la proportion observée tend vers 0.5 lorsque N augmente.
##  C'est une illustration de la loi des grands nombres.
```

Exercice 6 :

```
cat("\014")

# Question 1 :
xVector <- 1 / 1:50; print(xVector) # xVector <- 1 / (1:50) # plus clair malg
ré que 1:50 est prioritaire à /

##  [1] 1.00000000 0.50000000 0.33333333 0.25000000 0.20000000 0.16666667
##  [7] 0.14285714 0.12500000 0.11111111 0.10000000 0.09090909 0.08333333
## [13] 0.07692308 0.07142857 0.06666667 0.06250000 0.05882353 0.05555556
## [19] 0.05263158 0.05000000 0.04761905 0.04545455 0.04347826 0.04166667
## [25] 0.04000000 0.03846154 0.03703704 0.03571429 0.03448276 0.03333333
## [31] 0.03225806 0.03125000 0.03030303 0.02941176 0.02857143 0.02777778
## [37] 0.02702703 0.02631579 0.02564103 0.02500000 0.02439024 0.02380952
## [43] 0.02325581 0.02272727 0.02222222 0.02173913 0.02127660 0.02083333
## [49] 0.02040816 0.02000000

# Question 2 :

numberIteration <- c(50, 100, 1000)

yVector <- numeric(0)     # vecteur vide yVector <- c() : type NULL
for (i in numberIteration){
  yValue <- sum(1 / (1:i)) - log(i)
  yVector <- c(yVector, yValue)
  # print(yVector)  # Affichage intermédiaire
}
print(yVector)

## [1] 0.5871823 0.5822073 0.5777156

#Exercice 6 : version 2

# ============================================================
# Question 1 : Création du vecteur xVector
# ============================================================
cat("\014")

# Définir la taille du vecteur

N <- 50
```

```r
# Créer le vecteur : xVector[i] = 1/i
xVector <- 1 / (1:N)

# Afficher le vecteur
print(xVector)
```

```
##  [1] 1.00000000 0.50000000 0.33333333 0.25000000 0.20000000 0.16666667
##  [7] 0.14285714 0.12500000 0.11111111 0.10000000 0.09090909 0.08333333
## [13] 0.07692308 0.07142857 0.06666667 0.06250000 0.05882353 0.05555556
## [19] 0.05263158 0.05000000 0.04761905 0.04545455 0.04347826 0.04166667
## [25] 0.04000000 0.03846154 0.03703704 0.03571429 0.03448276 0.03333333
## [31] 0.03225806 0.03125000 0.03030303 0.02941176 0.02857143 0.02777778
## [37] 0.02702703 0.02631579 0.02564103 0.02500000 0.02439024 0.02380952
## [43] 0.02325581 0.02272727 0.02222222 0.02173913 0.02127660 0.02083333
## [49] 0.02040816 0.02000000
```

```r
# ============================================================
# Question 2 : Calcul de yVector_N pour N = 50, 100 et 1000
# ============================================================

# Définir les différentes valeurs de N à tester
numberIteration <- c(50, 100, 1000)

# Préallouer un vecteur numérique pour stocker les résultats
yVector <- numeric(length(numberIteration))

# Parcourir les différentes valeurs de N
for (j in seq_along(numberIteration)) {

  # Récupérer la valeur courante de N
  N <- numberIteration[j]

  # Calculer la somme : 1 + 1/2 + ... + 1/N
  harmonicSum <- sum(1 / (1:N))

  # Calculer yVector_N = somme - ln(N)
  yVector[j] <- harmonicSum - log(N)
}

# Afficher les résultats finaux
resultats <- data.frame(
  N = numberIteration,
  yVector_N = yVector
)

print(resultats)
```

```
##      N yVector_N
## 1   50 0.5871823
## 2  100 0.5822073
## 3 1000 0.5777156
```

```
# ============================================================
# Approximation de la limite lorsque N tend vers +infini
# ============================================================

# Une seule ligne de commande pour obtenir une approximation
sum(1 / (1:1000000)) - log(1000000)
```

```
## [1] 0.5772162
```

```
# La suite converge vers une constante appelée constante d'Euler-Mascheroni,
notée γ,
# dont la valeur approchée est : γ ≈ 0.57721566
```

Exercice 7 :

```
cat("\014")

piApproach <- 4 * sum((-1)^(0:100000) / ((2 * 0:100000) + 1))
print(pi)
```

```
## [1] 3.141593
```

```
print(piApproach)
```

```
## [1] 3.141603
```

```
print(abs(pi - piApproach))
```

```
## [1] 9.9999e-06
```

```
sprintf("%.6f", piApproach)
```

```
## [1] "3.141603"
```

```
sprintf("%.6f", pi)
```

```
## [1] "3.141593"
```

```
# Approximation de pi par la formule de Leibniz
cat("\014")

N <- 1000000

piApproach <- 4 * sum((-1)^(0:N) / (2 * (0:N) + 1))
```

```r
# Afficher l'approximation et la valeur de référence
print(piApproach)

## [1] 3.141594

print(pi)

## [1] 3.141593

# Afficher les six premières décimales
print(sprintf("%.6f", piApproach))

## [1] "3.141594"

print(sprintf("%.6f", pi))

## [1] "3.141593"

# Approximation de pi par la formule de Leibniz
# cat("\014")
# N <- 1
# piApproach <- 4 * sum((-1)^(0:N) / (2 * (0:N) + 1))
#
# # Augmenter N jusqu'à obtenir les 6 décimales correctes
# while (sprintf("%.6f", piApproach) != sprintf("%.6f", pi)) {
#   N <- N + 1000
#   piApproach <- 4 * sum((-1)^(0:N) / (2 * (0:N) + 1))
# }
#
# # Afficher les résultats
# print(N)
# print(piApproach)
# print(sprintf("%.6f", piApproach))

# très long

# Approximation de pi par la série de Leibniz
cat("\014")
```

```r
N <- 0
somme <- 0
piApproach <- 0

temps <- system.time({

  repeat {
    # Ajouter le terme correspondant à k = N
    somme <- somme + (-1)^N / (2 * N + 1)

    # Calculer l'approximation de pi
    piApproach <- 4 * somme
```

```
    # Vérifier les six décimales après arrondi
    if (sprintf("%.6f", piApproach) == sprintf("%.6f", pi)) {
      break
    }

    N <- N + 1
  }
})

# Afficher les résultats
cat("Nombre de termes :", N + 1, "\n")
```

```
## Nombre de termes : 1181461
```

```
cat("Approximation de pi :", piApproach, "\n")
```

```
## Approximation de pi : 3.141593
```

```
cat("Valeur de référence :", pi, "\n")
```

```
## Valeur de référence : 3.141593
```

```
cat("Six décimales obtenues :", sprintf("%.6f", piApproach), "\n")
```

```
## Six décimales obtenues : 3.141593
```

```
cat("Temps d'exécution (secondes) :", temps["elapsed"], "\n")
```

```
## Temps d'exécution (secondes) : 7.75
```

Exercice 8 :

notes <- c(12, 8, 15, 9, 17, 11, 6, 14, 10, 18, 7, 13, 16, 9, 12, 5, 14, 11, 19, 8)

Questions :

1. Calculer l'effectif, la moyenne, la médiane et l'écart-type.
2. Déterminer les notes minimale et maximale.
3. Compter les étudiants ayant obtenu une note supérieure ou égale à 10.
4. Calculer le pourcentage de réussite.
5. Extraire les notes strictement supérieures à la moyenne.
6. Trier les notes par ordre décroissant.
7. Calculer les premier et troisième quartiles.
8. Comparer la moyenne et la médiane. Que peut-on en déduire ?

```
cat("\014")
```

```
notes <- c(12, 8, 15, 9, 17, 11, 6, 14, 10,
           18, 7, 13, 16, 9, 12, 5, 14, 11, 19, 8)
```

```r
# Question 1 :
effectif <- length(notes); moyenne <- mean(notes)
mediane <- median(sort(notes)); ecartType <- sd(notes)
cat("L'effectif est    : ", sprintf("%5.3f", effectif), "\n",
    "La moyenne est    : ", sprintf("%5.3f", moyenne), "\n",
    "La médiane est    : ", sprintf("%5.3f", mediane), "\n",
    "L'écart-types est : ", sprintf("%6.3f", ecartType), "\n",
    sep = ""
    )
```

```
## L'effectif est    : 20.000
## La moyenne est    : 11.700
## La médiane est    : 11.500
## L'écart-types est :  4.028
```

```r
# Question 2 :
minMark <- min(notes); print(minMark)
```

```
## [1] 5
```

```r
maxMark <- max(notes); print(maxMark)
```

```
## [1] 19
```

```r
# Question 3 :
studentGreat10 <- sum(notes >= 10)
# cat("Le nombre d'étudiants ayant une notes >= 10 : ", format(studentGreat10,
 width = 4))
cat("Le nombre d'étudiants ayant une note >= 10 : ",
    format(studentGreat10, trim = TRUE), "\n")
```

```
## Le nombre d'étudiants ayant une note >= 10 :  13
```

```r
# Question 4 :

successPercent <- (studentGreat10 / effectif) * 100
cat("Le pourcentage de réussite est : ", successPercent, "%\n")
```

```
## Le pourcentage de réussite est :  65 %
```

```r
# Question 5 :
notesGreatMean <- notes[notes > moyenne]; print(notesGreatMean)
```

```
##  [1] 12 15 17 14 18 13 16 12 14 19
```

```r
# Question 6 :
sortedNotes <- sort(notes, decreasing = TRUE); print(sortedNotes)
```

```
##  [1] 19 18 17 16 15 14 14 13 12 12 11 11 10  9  9  8  8  7  6  5
```

```r
# Question 7 : # Calcul des quartiles
firstQuartile <- quantile(notes, 0.25)
```

```r
thirdQuartile <- quantile(notes, 0.75)

# Affichage des résultats
cat("Le premier quartile Q1 : ", firstQuartile, "\n")

## Le premier quartile Q1 :  8.75

cat("Le troisième quartile Q3 : ", thirdQuartile, "\n")

## Le troisième quartile Q3 :  14.25

# Question 8 :

# Comparaison
if (moyenne > mediane) {
  cat("La moyenne est supérieure à la médiane.\n")
} else if (moyenne < mediane) {
  cat("La moyenne est inférieure à la médiane.\n")
} else {
  cat("La moyenne est égale à la médiane.\n")
}

## La moyenne est supérieure à la médiane.

# Moyenne supérieure à la médiane : cela peut indiquer une distribution asymé
trique à droite, avec des valeurs élevées qui tirent la moyenne vers le haut.
```

Exercice 9 : Nettoyage d'un jeu de données

temperatures <- c(22.5, 23.1, NA, 24.0, 22.8, 150, 23.4, NA, 21.9, 22.7, 23.0)

Questions :

1. Détecter les valeurs manquantes avec is.na().
2. Compter les valeurs manquantes.
3. Construire un vecteur sans les valeurs manquantes.
4. Calculer la moyenne en présence de NA, puis avec na.rm = TRUE.
5. Détecter la valeur 150, qui paraît anormale dans ce contexte.
6. Calculer la moyenne et la médiane avant et après exclusion de cette valeur.
7. Proposer une règle de détection des valeurs aberrantes fondée sur l'intervalle interquartile.
8. Expliquer pourquoi une valeur aberrante ne doit pas être supprimée automatiquement.

```r
temperatures <- c(22.5, 23.1, NA, 24.0, 22.8,
                  150, 23.4, NA, 21.9, 22.7, 23.0)
# Question 1 :
manquantesValues <- is.na(temperatures); print(manquantesValues)
```

```
##  [1] FALSE FALSE  TRUE FALSE FALSE FALSE FALSE  TRUE FALSE FALSE FALSE
```

```
# Question 2 :
countNA <- sum(is.na(temperatures)); print(countNA)
```

```
## [1] 2
```

```
# Question 3 :
temperaturesWithoutNA <- temperatures[!is.na(temperatures)]; print(temperatur
esWithoutNA)
```

```
## [1]  22.5  23.1  24.0  22.8 150.0  23.4  21.9  22.7  23.0
```

```
temperaturesWithoutNA <- temperatures[-which(is.na(temperatures))]; print(tem
peraturesWithoutNA)
```

```
## [1]  22.5  23.1  24.0  22.8 150.0  23.4  21.9  22.7  23.0
```

```
# Question 4 :
moyenneNA <- mean(temperatures) ; print(moyenneNA)
```

```
## [1] NA
```

```
moyenne <- mean(temperatures, na.rm = TRUE); print(moyenne)
```

```
## [1] 37.04444
```